DAT 471 – Computational Techniques for Large-Scale Data

Module 4

Group 6

Name	Programme	Email
Gabriela Castillo Avila	MPHPC	avilag@student.chalmers.se
Mahnoor Ali Cheema	MPDSC	mahnoora@student.chalmers.se

20 May 2026

Problem 1 - Twitter Followers

The main difference between MRjob framework used in previous assignment and Spark used in this assignment is how computations are expressed and executed.

MRJobs, the computation follows a traditional MapReduce model with its phases defined as followed: Mapper → Shuffle\Sort → Reducer. The programmer manually defines the programs mapper and reduce operations.
In Spark, these computaions are instead expressed as transformations on the distributed dataset. The Spark framework builds a pipeline using trabsfromations such as map(), a filetr(), and reduceByKey(). These are executed in parallel across workers.
Huge data + Functional transformation + Parallel execution.

a

The written code followed a similar structure and midset as in assigment 3, following the Spark framework structure:

```
def user_following_count(line):
    user_id, follows = line.split(":",1)
    follows = follows.strip()
    user_id = user_id.strip()

    if follows == "":
        following_count = 0
    else:
        following_count = len([x for x in follows.split() if x.strip() != ""])  A
    return (user_id, following_count)

user_following_counts = lines.map(user_following_count).cache()
max_number_users = user_following_counts.max(key=lambda x: x[1])
total_following_all_users = user_following_counts.map(lambda x: x[1]).sum()
number_users = user_following_counts.count()
count_zero_following = user_following_counts.filter(lambda x: x[1] == 0).count()
average_following = total_following_all_users / number_users
end = time.time()

total_time = end - start
```

The code was executed in the apptainer using the following command:

```
python3 assignment4_problem1.py  -w 32 /data/.../twitter/twitter-2010_10M.txt
```

Number of cores	Runtime	Speedup
1	28.087s	1
2	17.670s	1.59
4	12.738s	2.20
8	9.953s	2.82
16	9.821	2.86
32	10.327	2.72
64	10.248s	2.74

Table 1: Following execution time results using Spark

b

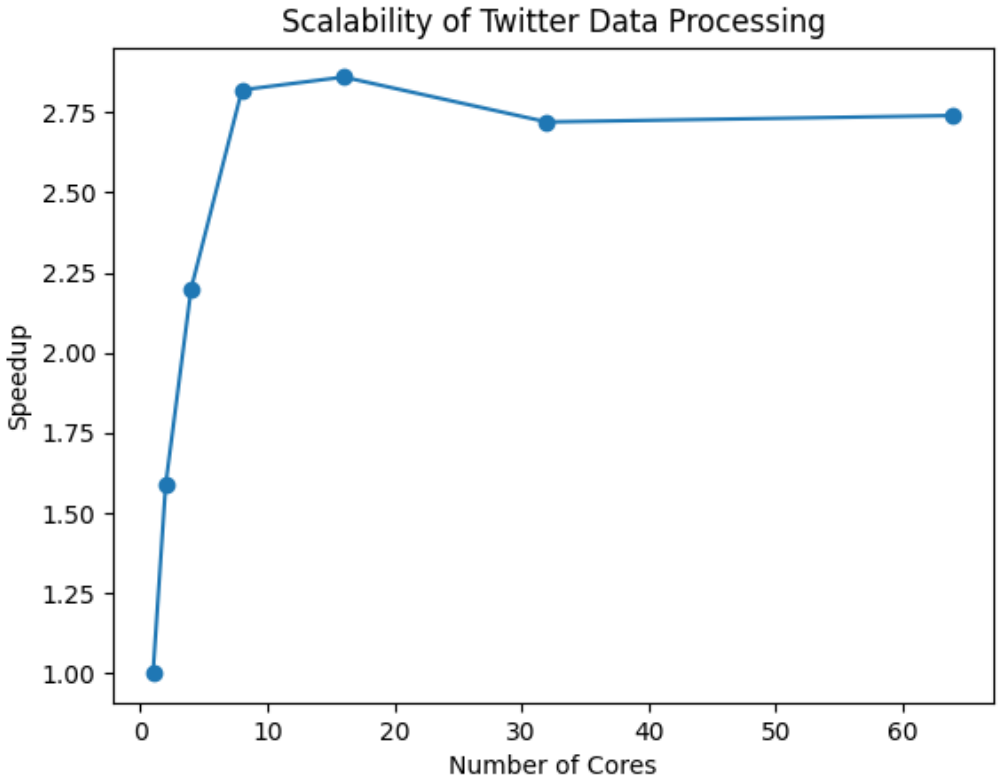


Figure 1: Following runtime speedup

Figure 1 shows the plot of the speedup values from table 1.

Output for a single core execution:

```
max follows: 15131310 follows 119880
users follow on average: 8.36385
number of user who follow no-one: 3754459
num workers: 1
total time: 28.087105224609375
```

c

The following code was used to gather the followers information using Spark:

```
def follower_count(line):
    user_id, followers = line.split(":",1)
    followers = followers.strip()

    if followers == "":
        return []

    followed = followers.split()
    return [(user_id, followed_user) for followed_user in followed]


user_followers = lines.flatMap(follower_count)
follower_counts = user_followers.map(lambda x: (x[1], 1)).
reduceByKey(lambda a, b: a + b).cache()

max_number_users = follower_counts.max(key=lambda x: x[1])
total_users = lines.map(lambda line: line.split(":",1)[0]).cache()
total_followers_all_users = follower_counts.map(lambda x: x[1]).sum()
number_users = total_users.count()
count_zero_followers = number_users - follower_counts.count()
average_followers = total_followers_all_users / number_users
```

Number of cores	Runtime	Speedup
1	124.880	1
2	65.786	1.90
4	40.514	3.08
8	26.785	4.66
16	21.129	5.91
32	19.111	6.53
64	18.035	6.92

Table 2: Followers execution time results using Spark

d

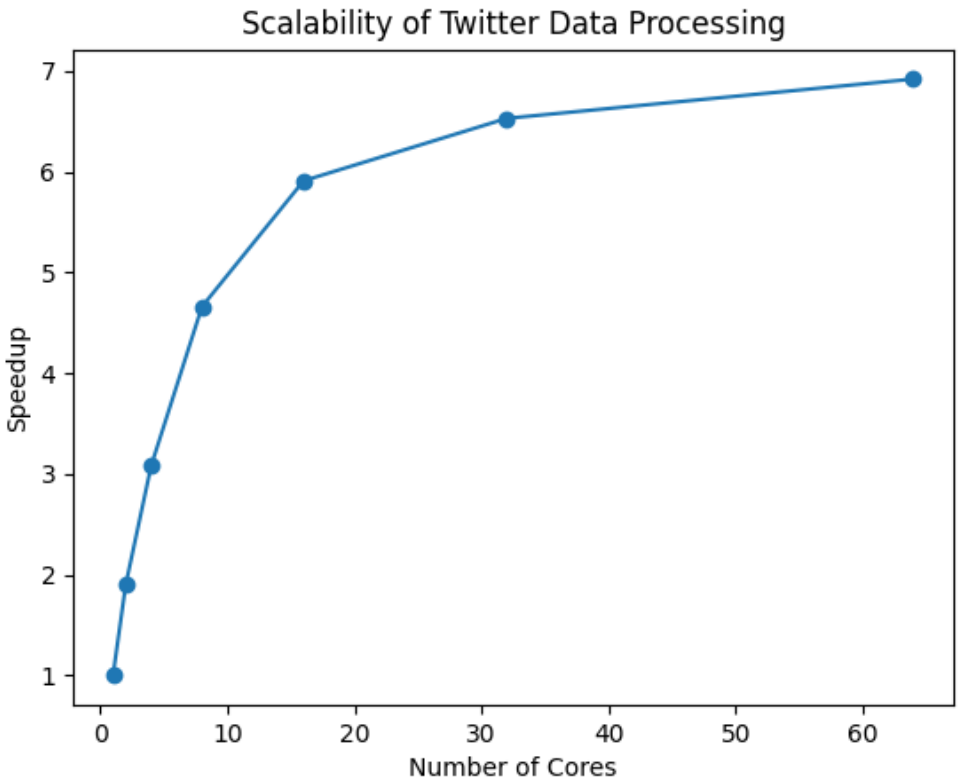


Figure 2: Follower runtime speedup

Figure 2 shows the plot of the speedup values from table 2.
Output for a single core execution:

```
max followers: 19757371 has 443107 followers
followers on average: 8.3638526
number of user with no followers: 2485440
num workers: 1
total time: 124.87977766990662
```

Problem B - Climate

Part a

1- Julian date per record

Following line of code computes Julian date per record

```
df = df.withColumn("JDN", jdn(col("DATE")))
```

2- Compute $T_{avg} = T_{min} + T_{max}/2$

It is done as follows:

```
df = df.withColumn("Tavg", (col("TMIN") + col("TMAX")) / 2)
```

3- Per station linear regression T_{avg} vs JDN(least squares)

It is computed as follows:

```
slopes = df.groupBy("station").apply(lsq)
```

4- Print top 5 stations by slope(code, name, slope in °F/d)

Station Code	Station Name	Slope (°F/d)
MXN00005023	PALESTINA DGE, MX	1.361×10^{-3}
MXN00009052	UNIDAD MODELO, MX	9.292×10^{-4}
MXN00009003	AQUILES SERDAN 46, MX	3.606×10^{-4}
SWE00139268	FREDRIKSBERG, SW	2.556×10^{-4}
CA003052995	HAILSTONE BUTTE LO, AB CA	2.038×10^{-4}

Table 3: Top 5 stations with the greatest warming slopes (unit: °F/d).

5- Five number summary of slopes (°F/d)

We computed for climate_small.csv and observed that it matches with "small" row in table 4 of assignment, following is the output we got:

Statistic	Value (°F/d)
Min	-1.250×10^{-3}
Q1	-3.604×10^{-6}
Median	2.721×10^{-5}
Q3	1.206×10^{-4}
Max	1.361×10^{-3}

Table 4: Five-number summary of slope coefficients (unit: °F/d).

6- Fraction of positive slopes

Computed as follows:

```
    total = slopes_clean.count()
positive = slopes_clean.filter(col("slope") > 0).count()
frac_positive = positive / total
```

and the output we got is:

```
Fraction of positive coefficients:
0.7105263157894737
```

7- Decadewise differences (2010s vs 1910s) for stations with both decades

It is implmented as follows:

```
 df = df.withColumn("DATE", to_date(col("DATE")))   # convert string → date
df = df.withColumn("YEAR", year(col("DATE")))      # extract YEAR
df = df.withColumn("decade", (col("YEAR") / 10).cast(IntegerType()) * 10)
```

8- Top 5 stations by positive decadewise difference (°C)

Station Code	Station Name	Difference (°C)
USC00313976	HENDERSONVILLE 1 NE, NC US	1.0
USC00398472	TYNDALL, SD US	0.7
USC00410493	BALLINGER 2 NW, TX US	0.3
USC00401790	CLARKSVILLE WWTP, TN US	0.1
USC00144712	LINCOLN 1 SE, KS US	0.1

Table 5: Top 5 stations with the greatest positive decadewise temperature differences (unit: °C).

9- Five number summary of temperature differences (°C)

Below is the summary for small dataset:

Statistic	Value (°C)
Min	-0.5
Q1	0.1
Median	0.1
Q3	0.7
Max	1.0

Table 6: Five-number summary of decadewise temperature differences (unit: °C).

10- Fraction of positive temperature differences

Following is the output we got:

```
Fraction of positive differences:
0.8333333333333334
```

Part b - Plot the speedup as the function of CPU cores for w = 1, 2, 4, . . . , 64 workers

Firstly, following is the time in seconds each worker is taking to do computations on large dataset.

Workers (w)	Time T(w) [s]
1	2540.4
2	1340.5
4	717.7
8	702.4
16	367.2
32	305.0
64	300.7

Table 7: Execution time for the large dataset as a function of number of workers.

Now, we compute speedup using the following formula:

$$S(w) = \frac{T(1)}{T(w)}$$

And, following is the plot between speedup S(w) and number of workers (w).

Workers (w)	Time T(w) [s]	Speedup S(w)
1	2540.4	1.00
2	1340.5	1.89
4	717.7	3.54
8	702.4	3.62
16	367.2	6.92
32	305.0	8.33
64	300.7	8.45

Table 8: Execution time and speedup for the large dataset.

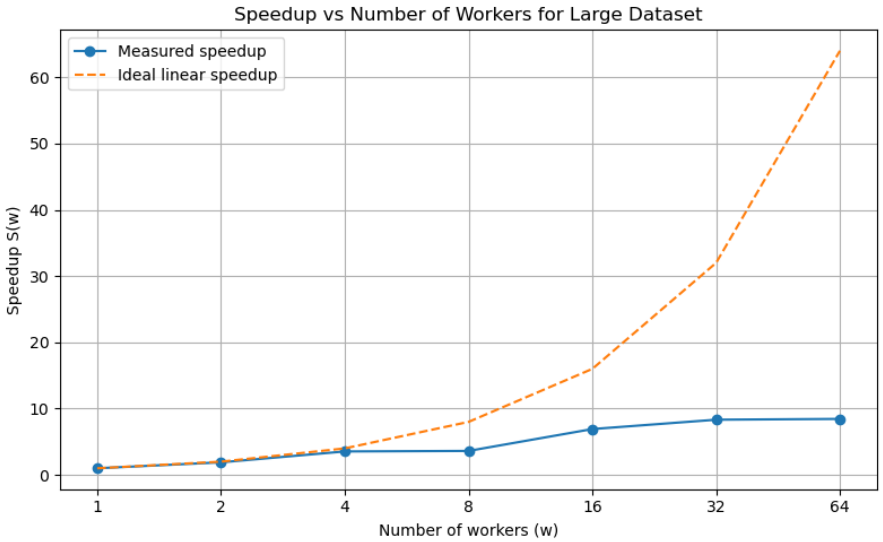


Figure 3: Speedup as a function of number of workers for the large dataset.

Part c - Time spent in Reading and Computation - Calculate the speedup for computations using 64 cores

First we computed read and computation time for 1 core and then for 64 cores and their respective fractions are computed using the following formula :

$$Readfraction(w) = \frac{T_{read}(w)}{T_{total}(w)}$$

$$Computefraction(w) = \frac{T_{compute}(w)}{T_{total}(w)}$$

Therefore, the speedup computation using 64 cores becomes:

$$S_{compute}(64) = \frac{T_{compute}(1)}{T_{compute}(64)}$$

Time Breakdown and Computation-Only Speedup

From the implentation, we obtained the following timings for the large dataset:

$$T_{\text{read}}(1) = 130.9 \text{ s}, \quad T_{\text{compute}}(1) = 2182.9 \text{ s},$$
$$T_{\text{read}}(64) = 17.3 \text{ s}, \quad T_{\text{compute}}(64) = 277.0 \text{ s}.$$

The total times are therefore:

$$T_{\text{total}}(1) = 2313.8 \text{ s}, \qquad T_{\text{total}}(64) = 294.3$$

Fraction of time spent reading.

$$\text{Read fraction}(1) = \frac{130.9}{2313.8} \approx 0.057 \quad (5.7\%),$$

$$\text{Read fraction}(64) = \frac{17.3}{294.3} \approx 0.059 \quad (5.9\%).$$

Fraction of time spent computing.

$$\text{Compute fraction}(1) = \frac{2182.9}{2313.8} \approx 0.943 \quad (94.3\%),$$

$$\text{Compute fraction}(64) = \frac{277.0}{294.3} \approx 0.941 \quad (94.1\%).$$

Thus, in both configurations, roughly 94% of the total runtime is spent in computation, while only about 6% is spent reading the data.

Computation-only speedup at 64 workers.

The computation-only speedup is defined as:

$$S_{\text{compute}}(64) = \frac{T_{\text{compute}}(1)}{T_{\text{compute}}(64)} = \frac{2182.9}{277.0} \approx 7.88.$$

$$\boxed{S_{\text{compute}}(64) \approx 7.9\times}$$

This shows that the computational part of the workload scales reasonably well, even though the overall speedup is limited by the fixed cost of reading the data.

Part d - Results for the Full Dataset

Top 5 slopes

Station	Name	Slope (°F/day)
USC00482175	CRESTON, WY US	2.817×10^{-2}
CA005062730	SOUTH BAY, MB CA	2.252×10^{-2}
USC00104910	KILGORE, ID US	1.244×10^{-2}
USC00506227	STERLING, AK US	8.704×10^{-3}
USC00398572	VAN METRE, SD US	6.979×10^{-3}

Table 9: Top 5 slope coefficients for the full dataset

Fraction of positive slopes = 0.7453

Five-number summary of slopes

$$\begin{aligned}
\beta_{\text{min}} &= -2.649 \times 10^{-1} \\
\beta_{Q1} &= -1.242 \times 10^{-6} \\
\beta_{\text{median}} &= 6.266 \times 10^{-5} \\
\beta_{Q3} &= 1.380 \times 10^{-4} \\
\beta_{\text{max}} &= 2.817 \times 10^{-2}
\end{aligned}$$

Top 5 decade temperature differences (1910s vs 2010s)

Fraction of positive differences = 0.7520

Station	Name	Difference (°C)
RSM00024266	VERHOJANSK, RS	36.1
KZ000028676	PETROPAVLOVSK, KZ	17.2
RSM00029612	BARABINSK, RS	15.5
USC00301974	DANSVILLE, NY US	12.5
USC00179891	WOODLAND, ME US	11.9

Table 10: Top 5 decade temperature differences for the full dataset

Five-number summary of decade differences

$$\begin{aligned} d_{\min} &= -16.4\ ^\circ\mathrm{C} \\ d_{Q1} &= 0.0\ ^\circ\mathrm{C} \\ d_{\text{median}} &= 0.8\ ^\circ\mathrm{C} \\ d_{Q3} &= 1.6\ ^\circ\mathrm{C} \\ d_{\max} &= 36.1\ ^\circ\mathrm{C} \end{aligned}$$

Total wall-clock time and workers used

$$T_{\text{total}} = 301.4\ \text{s}, \qquad w = 64.$$

Part e - Interpretation of the Results

Although the dataset contains unsanitized and potentially erroneous historical measurements, the overall statistical patterns are clear. The five-number summary of the slope coefficients shows that the median trend is positive and that approximately 74.5% of all stations exhibit a positive slope. This indicates that, for most stations, daily average temperatures have increased over time. The upper quartile and maximum slope values are also distinctly positive, suggesting that a substantial subset of stations experiences relatively strong warming trends.

The decade-wise temperature differences reinforce this picture. The median difference between the 1910s and 2010s is 0.8°C, and the upper quartile is 1.6°C, with about 75.2% of stations showing a positive difference. Even though the dataset contain some extreme or questionable values (e.g, very large positive or negative differences), the central tendency remains robustly positive.

Overall, these results indicate that the majority of stations in the dataset have warmed over the last century. Despite noise and outliers in the raw data, the aggregated statistics provide a consistent signal of long-term temperature increase, aligning with the broader scientific understanding of global climate warming.